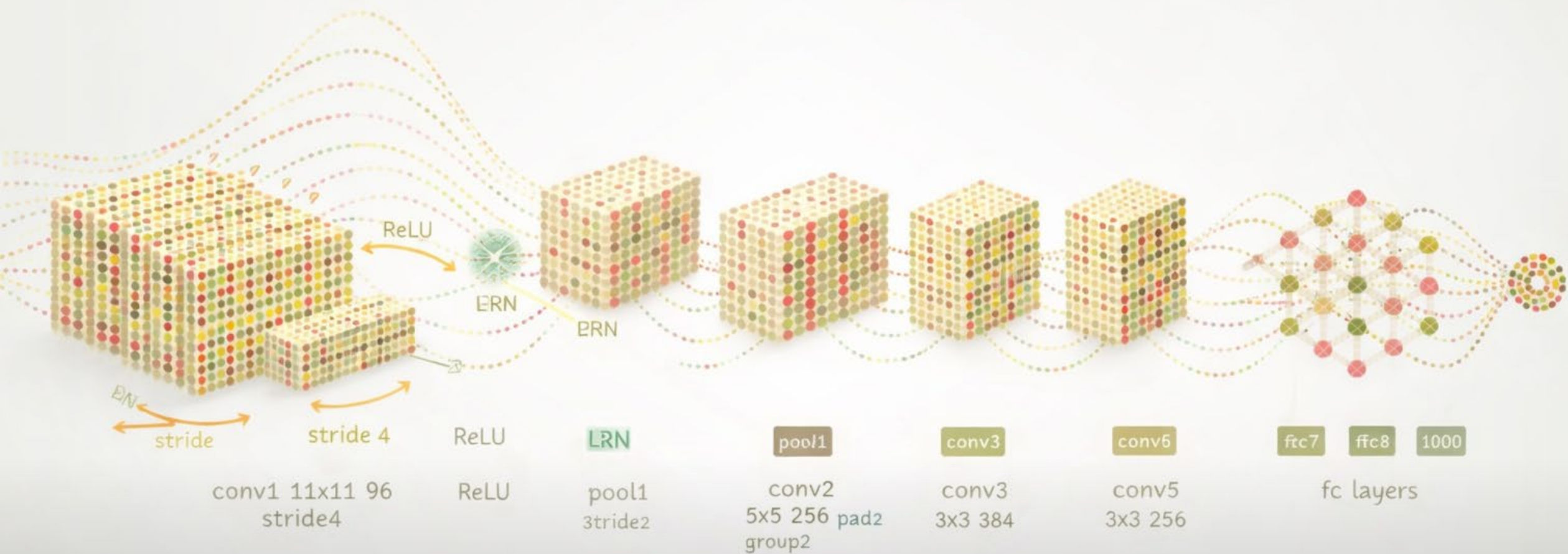




# CONVOLUTIONAL NEURAL NETWORKS

---

# TERMS

- ANN = Artificial neural network
- DNN = Deep neural network
- CNN = Convolutional neural network
- RNN = Recurrent neural network

---

# CONVOLUTIONAL NEURAL NETWORKS

- CNNs mimic the structure and function of the visual system
- They use layers of “convolutions” (or filters) to learn and extract features of the input data
- Specifically designed to process and analyze data with a grid-like structure
  - Images
  - Videos

---

# WHAT IS A CONVOLUTION?

- Mathematical operation on two functions ( $f$  and  $g$ ) that produces a third function ( $f * g$ ) that expresses how the shape of one is modified by the other

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

- Convolution is fancy multiplication
  - Convolutions are often described as multiplication and addition over a flipped sliding window
-

---

# CONTINUOUS EXAMPLE

- **Define two simple functions:**

$$f(t) = \begin{cases} 1 & 0 \leq t \leq 1 \\ 0 & \text{otherwise} \end{cases} \quad g(t) = \begin{cases} 1 & 0 \leq t \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

- **Convolution definition:**

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

- $\tau$  :integration variable
  - $t$ : output location
  - $g(t - \tau)$  :flipped and shifted version of  $g$
-



---

# CONTINUOUS EXAMPLE

- $f(\tau)$  is nonzero on  $[0, 1]$
- $g(t - \tau)$  is nonzero when  $t - 1 \leq \tau \leq t$
- **Therefore:**

$(f * g)(t) = \text{Length of Overlap between the intervals } [0, 1] \text{ and } [t - 1, t]$

---

---

# DISCRETE EXAMPLE

- Suppose patients come to our hospital this week with a certain disease and a different number of patients come each day
    - M: 1 patients  
T: 2 patients  
W: 3 patients  
Th: 4 patients  
F: 5 patient
    - In vector format: [1, 2, 3, 4, 5]
  - Suppose that we treat each patient with 3 doses of medication
  - How many doses of medication will we need each day?
    - [3, 6, 9, 12, 15]
-

---

# DISCRETE EXAMPLE

- Suppose patients come to our hospital this week with a certain disease and a different number of patients come each day
    - In vector format:  $[1, 2, 3, 4, 5]$
  - Now suppose that patients need a multi-day regimen with 3 doses the first day, 2 the second day, and 3 doses the third day
    - $[3, 2, 1]$
  - How many doses of medication will we need each day?
-



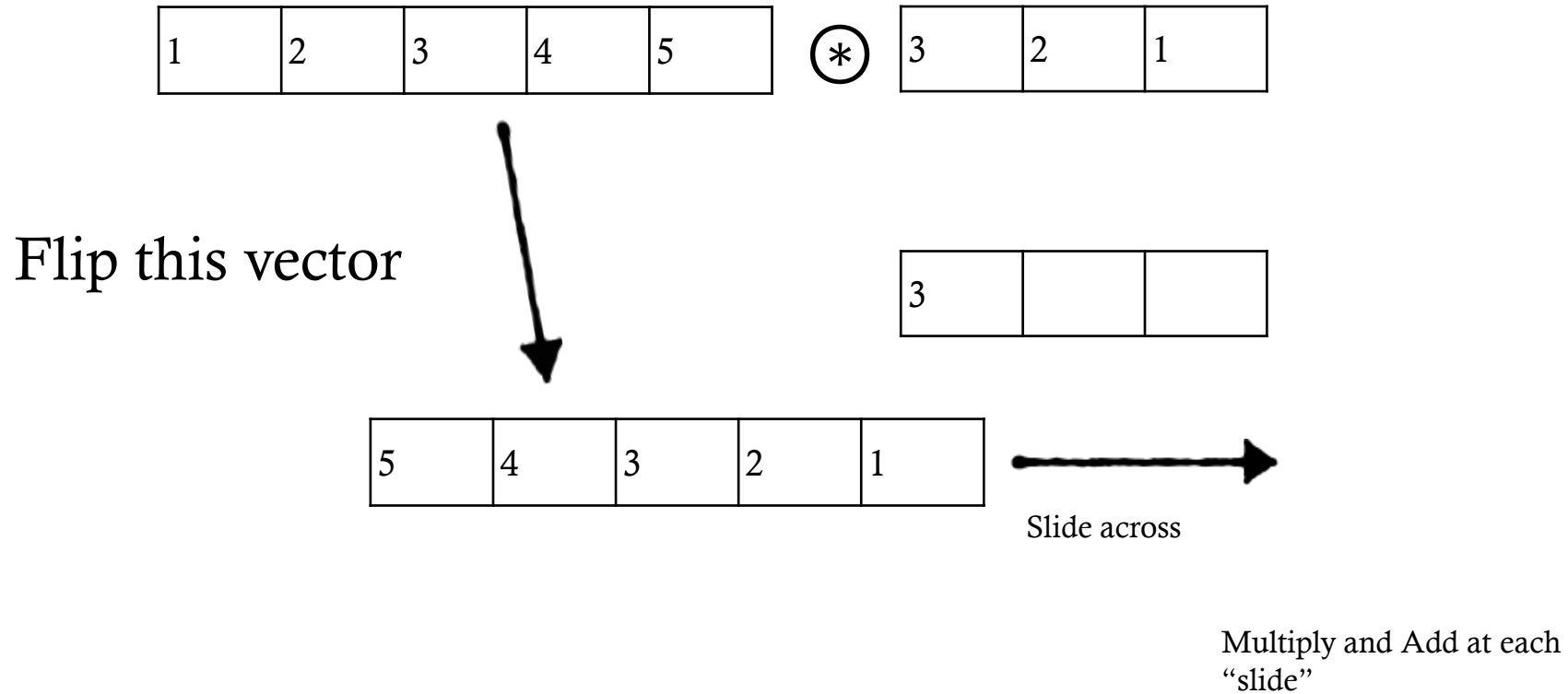
---

# DISCRETE EXAMPLE

- Suppose patients come to our hospital this week with a certain disease and a different number of patients come each day
    - In vector format: [1, 2, 3, 4, 5]
  - Now suppose that patients actually need a multi-day regimen with 3 doses the first day, 2 the second day, and 1 doses the third day
    - [3, 2, 1]
  - How many doses of medication will we need each day?
    - Monday:  $1*3$
    - Tuesday:  $2*3 + 1*2$
    - Wednesday:  $3*3 + 2*2 + 1*1$
    - Thursday:  $4*3 + 3*2 + 2*1$
    - Friday:  $5*3 + 4*2 + 3*1$
    - Saturday:  $5*2 + 4*1$
    - Sunday:  $5*1$
-

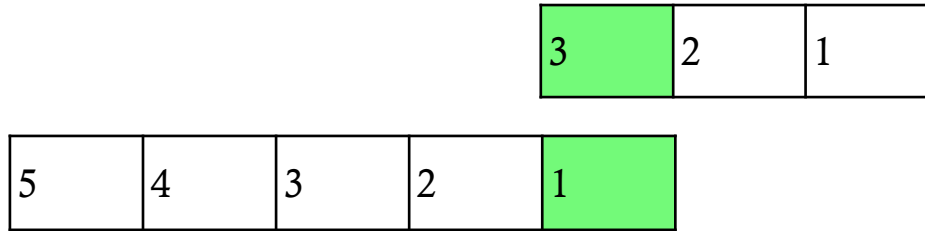
---

# DISCRETE EXAMPLE



---

# DISCRETE EXAMPLE



---

$$\begin{array}{|c|c|c|} \hline 3 & & \\ \hline \end{array} = \begin{array}{|c|} \hline 3 \\ \hline \end{array}$$

---

# DISCRETE EXAMPLE

|   |   |   |
|---|---|---|
| 3 | 2 | 1 |
|---|---|---|

|   |   |   |   |   |
|---|---|---|---|---|
| 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|

---

|   |   |  |
|---|---|--|
| 6 | 2 |  |
|---|---|--|

 $=$ 

|   |
|---|
| 8 |
|---|

---

# DISCRETE EXAMPLE

|   |   |   |   |   |
|---|---|---|---|---|
|   |   | 3 | 2 | 1 |
| 5 | 4 | 3 | 2 | 1 |

---

|   |   |   |
|---|---|---|
| 9 | 4 | 1 |
|---|---|---|

 $=$ 

|    |
|----|
| 14 |
|----|

---

---

# DISCRETE EXAMPLE

|   |   |   |   |   |
|---|---|---|---|---|
|   | 3 | 2 | 1 |   |
| 5 | 4 | 3 | 2 | 1 |

---

|    |   |   |
|----|---|---|
| 12 | 6 | 2 |
|----|---|---|

 $=$ 

|    |
|----|
| 20 |
|----|

---

---

# DISCRETE EXAMPLE

|   |   |   |
|---|---|---|
| 3 | 2 | 1 |
|---|---|---|

|   |   |   |   |   |
|---|---|---|---|---|
| 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|

---

|    |   |   |
|----|---|---|
| 15 | 8 | 3 |
|----|---|---|

 = 

|    |
|----|
| 26 |
|----|

---

# DISCRETE EXAMPLE

|   |   |   |
|---|---|---|
| 3 | 2 | 1 |
|---|---|---|

|   |   |   |   |   |
|---|---|---|---|---|
| 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|

---

|  |    |   |
|--|----|---|
|  | 10 | 4 |
|--|----|---|

 $=$ 

|    |
|----|
| 14 |
|----|

---



---

# DISCRETE EXAMPLE

|   |   |   |
|---|---|---|
| 3 | 2 | 1 |
|---|---|---|

|   |   |   |   |   |
|---|---|---|---|---|
| 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|

---

|  |  |   |
|--|--|---|
|  |  | 5 |
|--|--|---|

 = 

|   |
|---|
| 5 |
|---|

---

---

# DISCRETE EXAMPLE

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|

 $\circledast$ 

|   |   |   |
|---|---|---|
| 3 | 2 | 1 |
|---|---|---|

 =

|   |   |    |    |    |    |   |
|---|---|----|----|----|----|---|
| 3 | 8 | 14 | 20 | 26 | 14 | 5 |
|---|---|----|----|----|----|---|

```
np.convolve([1,2,3,4,5],[3,2,1])  
array([ 3,  8, 14, 20, 26, 14,  5])
```

---

# USES OF CONVOLUTIONS

- Convolutions have application in a vast array of disciplines
  - Probability
  - Statistics
  - Image processing
  - Signal processing
  - Acoustics
  - Computer vision
  - Etc.

---

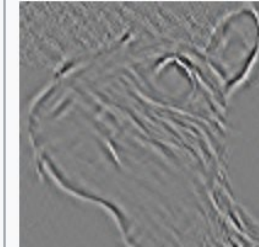
# CONVOLUTIONS IN IMAGE PROCESSING

- In image processing, convolving a filter with the original image results in a range of effects, depending on the filter



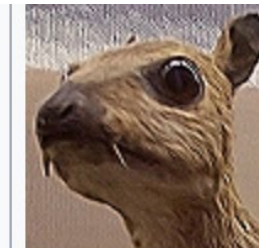
Edge detection

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



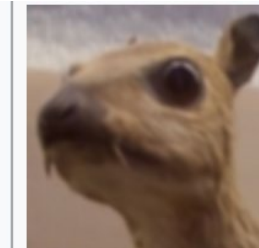
Sharpen

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Blur

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$



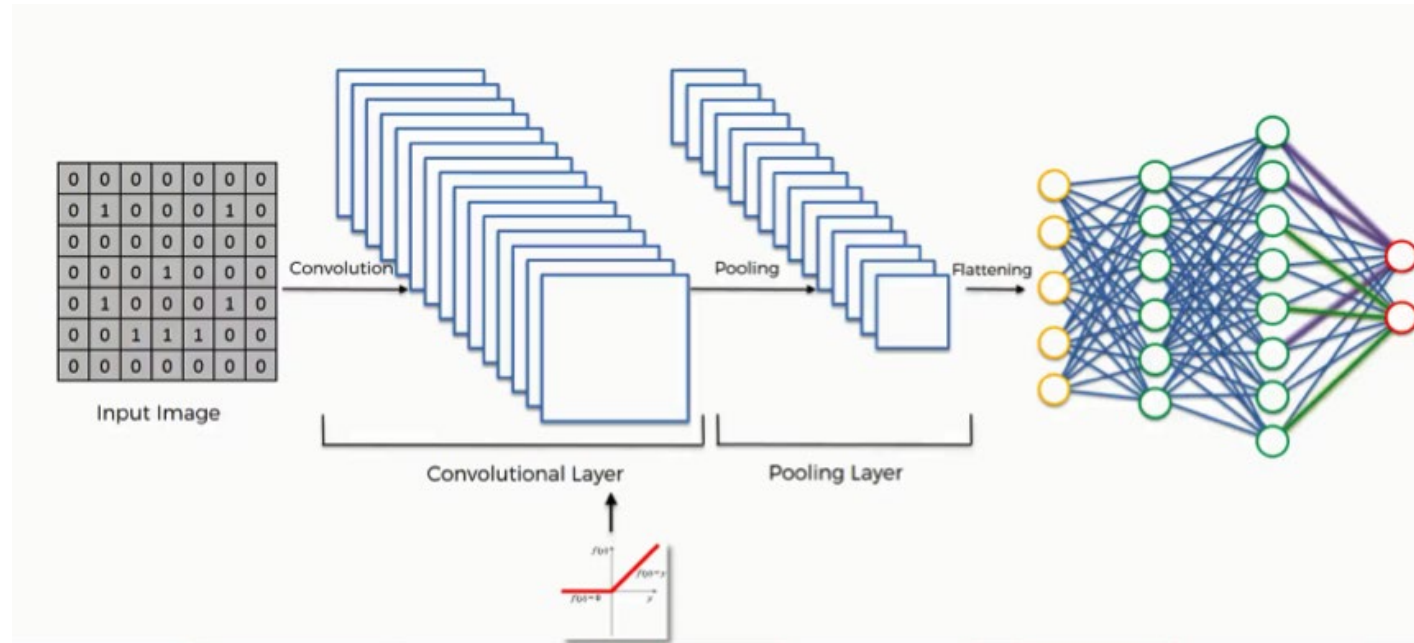
---

# CNN ARCHITECTURE

- Basic structure of a CNN:
  - Convolutional layers
    - part of fitting process involves the network finding the filters that help identify the image
  - Pooling layers
  - Flattening and Fully connected layers

---

# CNN ARCHITECTURE



---

# CONVOLUTIONAL LAYER

---

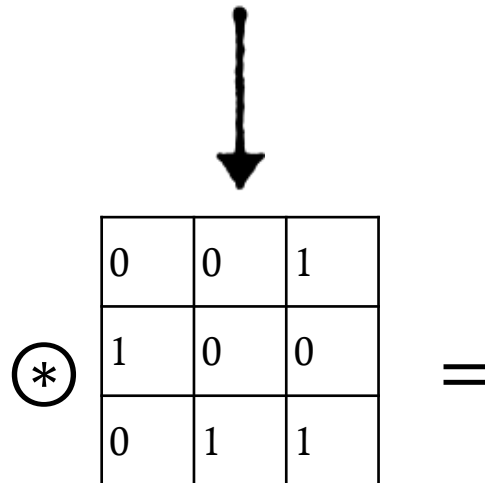
---

# CONVOLUTION WITH ONE FILTER

This is learned by the model

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image



Feature Detector/  
Filter

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

Feature Map

---



---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |  |  |  |  |
|---|--|--|--|--|
| 0 |  |  |  |  |
|   |  |  |  |  |
|   |  |  |  |  |
|   |  |  |  |  |
|   |  |  |  |  |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |  |  |  |
|---|---|--|--|--|
| 0 | 0 |  |  |  |
|   |   |  |  |  |
|   |   |  |  |  |
|   |   |  |  |  |
|   |   |  |  |  |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |  |  |
|---|---|---|--|--|
| 0 | 0 | 1 |  |  |
|   |   |   |  |  |
|   |   |   |  |  |
|   |   |   |  |  |
|   |   |   |  |  |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |  |
|---|---|---|---|--|
| 0 | 0 | 1 | 0 |  |
|   |   |   |   |  |
|   |   |   |   |  |
|   |   |   |   |  |
|   |   |   |   |  |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 |   |   |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---



---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 |   |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
|   |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 |   |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 |   |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 |   |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 |   |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
|   |   |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 |   |   |   |   |
|   |   |   |   |   |

Feature Map

---



---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 |   |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 |   |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 |   |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
|   |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 |   |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 |   |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 |   |   |

Feature Map

---

---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 |   |

Feature Map

---



---

# CONVOLUTION WITH ONE FILTER

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 1 | 1 | 0 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Input Image

$\otimes$

|   |   |   |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

Feature Detector/  
Filter

=

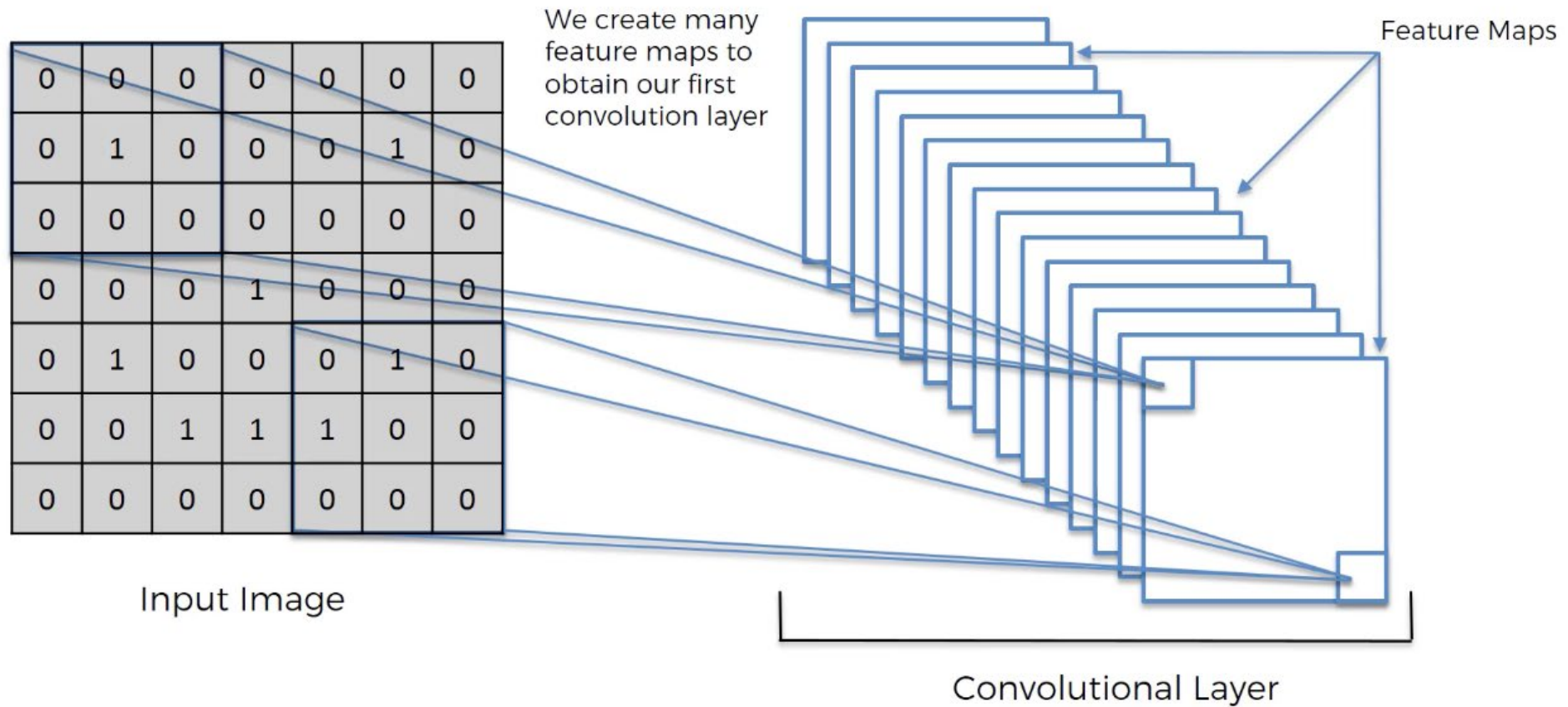
|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

---

---

# CONVOLUTIONAL LAYER



---

# POOLING LAYER

---

---

# MAX POOLING

- Pooling layers operate on the feature maps to summarize the pixels in a rectangular neighborhood
- Max pooling is the most common, though other pooling operations are occasionally used
- The purpose of the pooling layer is to:
  - Reduce the the spatial size of the feature map
  - Help make the network robust to small translations and distortions of the input data

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
→  
Box size: 2x2  
Stride = 2

|   |  |  |
|---|--|--|
| 1 |  |  |
|   |  |  |
|   |  |  |

Pooled Feature Map

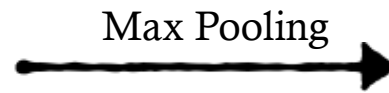
---

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map



Box size: 2x2  
Stride = 2

|   |   |  |
|---|---|--|
| 1 | 2 |  |
|   |   |  |
|   |   |  |

Pooled Feature Map

---

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
→  
Box size: 2x2  
Stride = 2

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
|   |   |   |
|   |   |   |

Pooled Feature Map

---

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
→  
Box size: 2x2  
Stride = 2

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 4 |   |   |
|   |   |   |

Pooled Feature Map

---



---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
→  
Box size: 2x2  
Stride = 2

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 4 | 2 |   |
|   |   |   |

Pooled Feature Map

---

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
Box size: 2x2  
Stride = 2



|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 4 | 2 | 1 |
|   |   |   |

Pooled Feature Map

---

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
Box size: 2x2  
Stride = 2



|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 4 | 2 | 1 |
| 0 |   |   |

Pooled Feature Map

---

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
→  
Box size: 2x2  
Stride = 2

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 4 | 2 | 1 |
| 0 | 2 |   |

Pooled Feature Map

---

---

# MAX POOLING

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 2 | 0 | 0 |
| 1 | 0 | 0 | 2 | 1 |
| 1 | 4 | 2 | 1 | 0 |
| 0 | 0 | 1 | 2 | 1 |

Feature Map

Max Pooling  
→  
Box size: 2x2  
Stride = 2

|   |   |   |
|---|---|---|
| 1 | 2 | 1 |
| 4 | 2 | 1 |
| 0 | 2 | 1 |

Pooled Feature Map

---

---

# FLATTENING & FULLY CONNECTED LAYERS

---

---

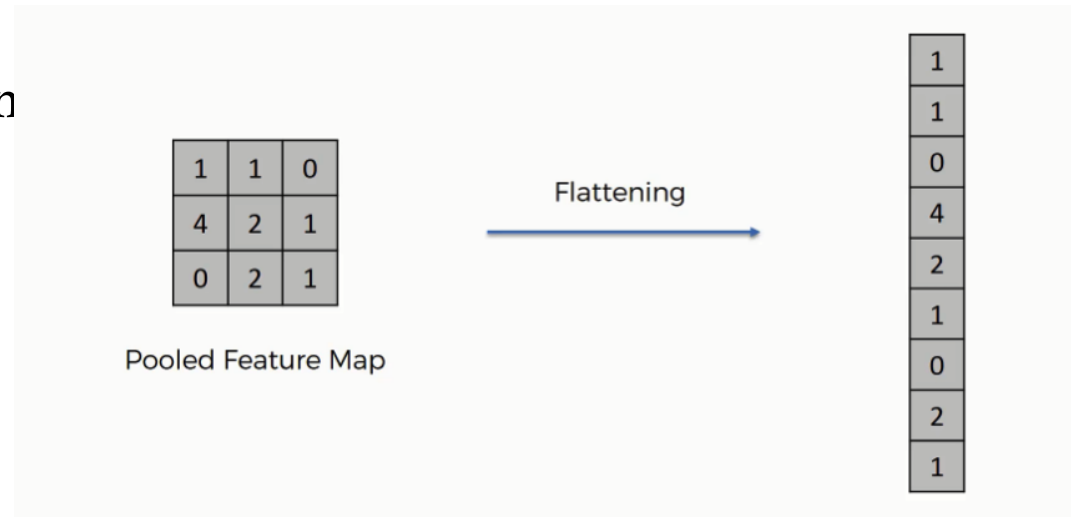
# IMAGE TO CLASSIFICATION

- After one or more convolution and pooling layers
- Send the result into a regular ANN
  - Combine features into a wider variety of attributes
  - Perform the classification step

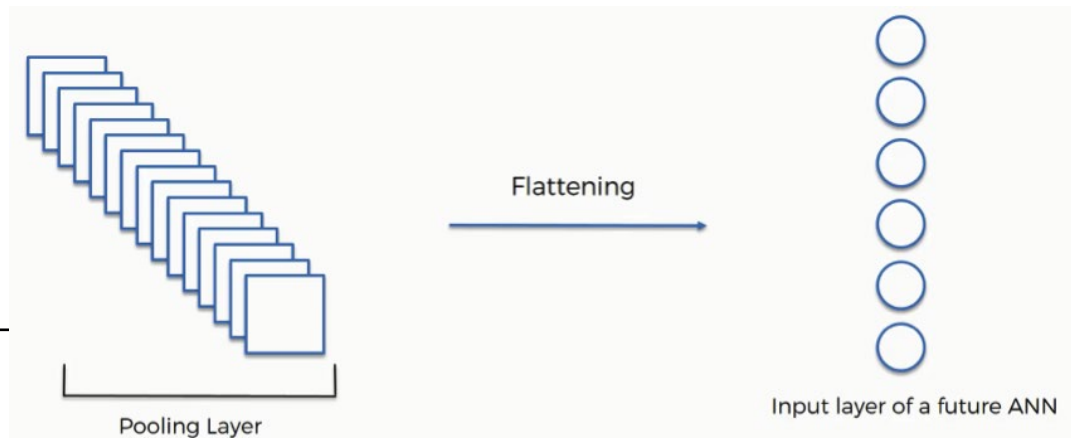
---

# FLATTENING

- Flattening simply mean  
ANN



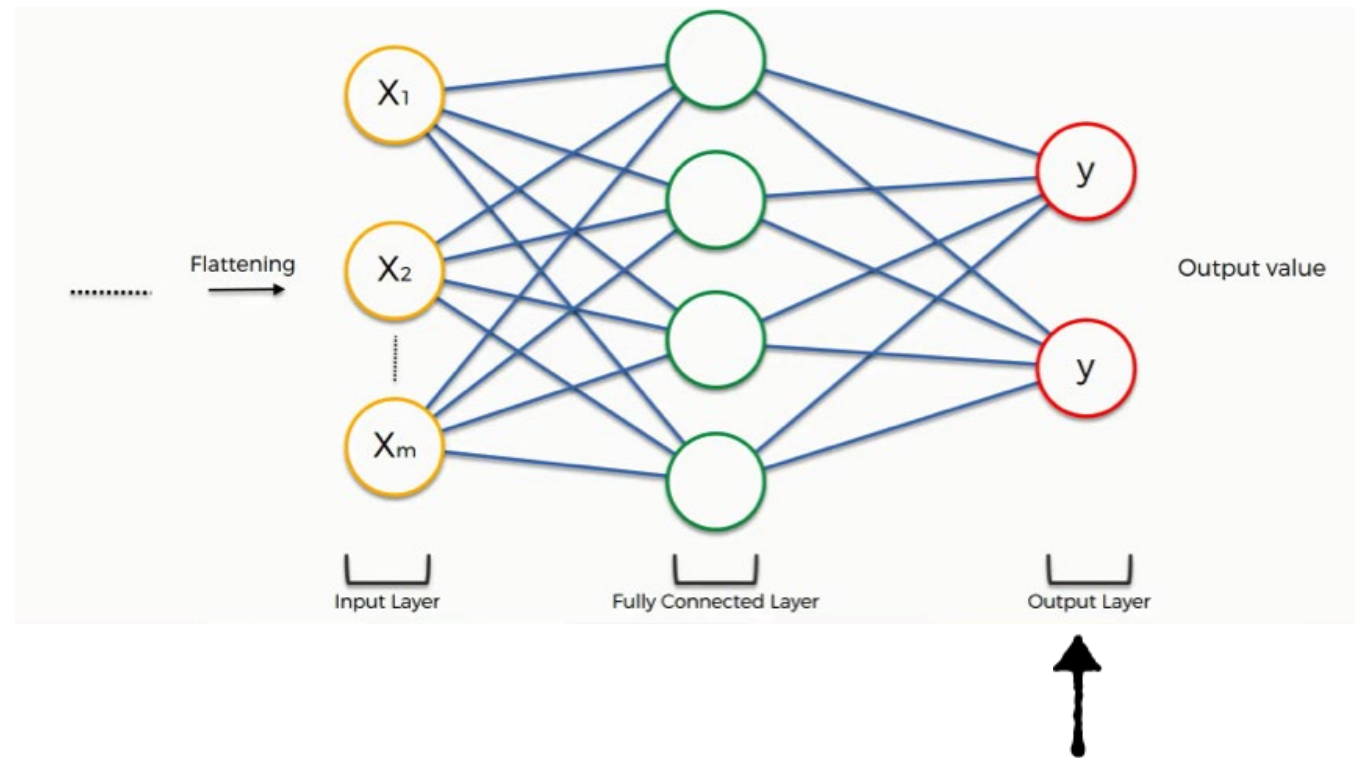
input into a regular





---

# FULLY CONNECTED LAYER

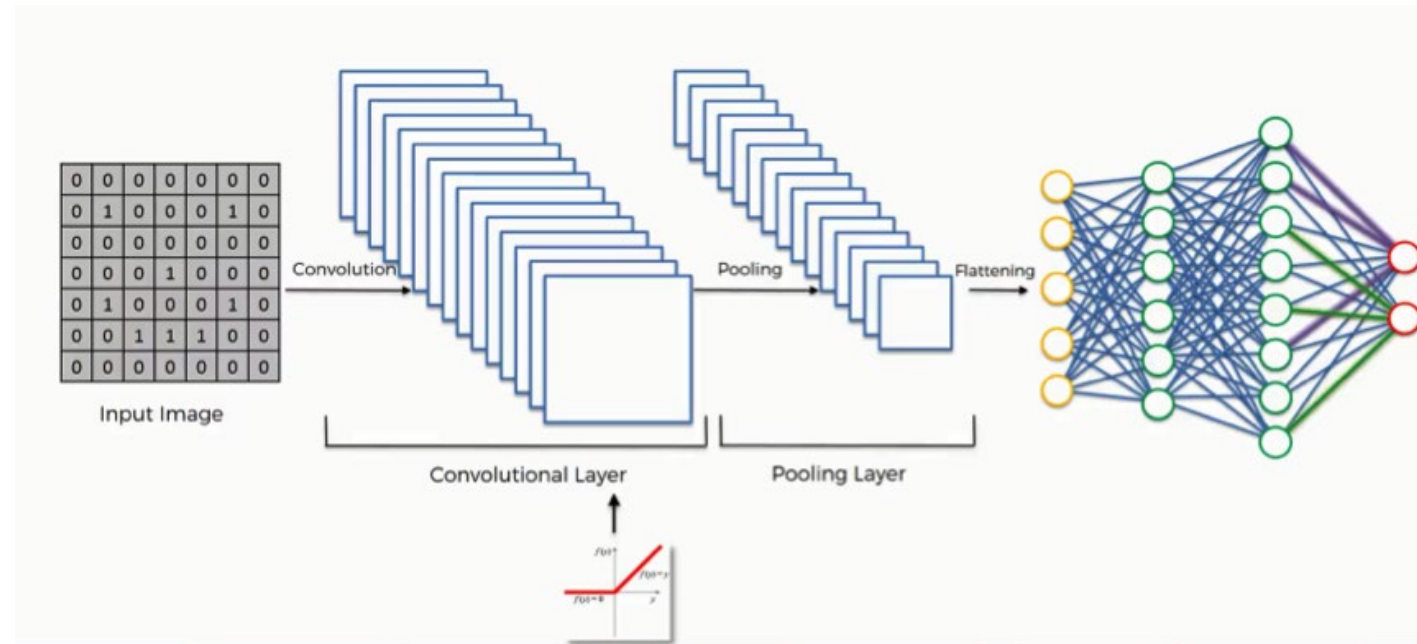


---

The output layer will have a neuron for every possible class

---

# SUMMARY



---

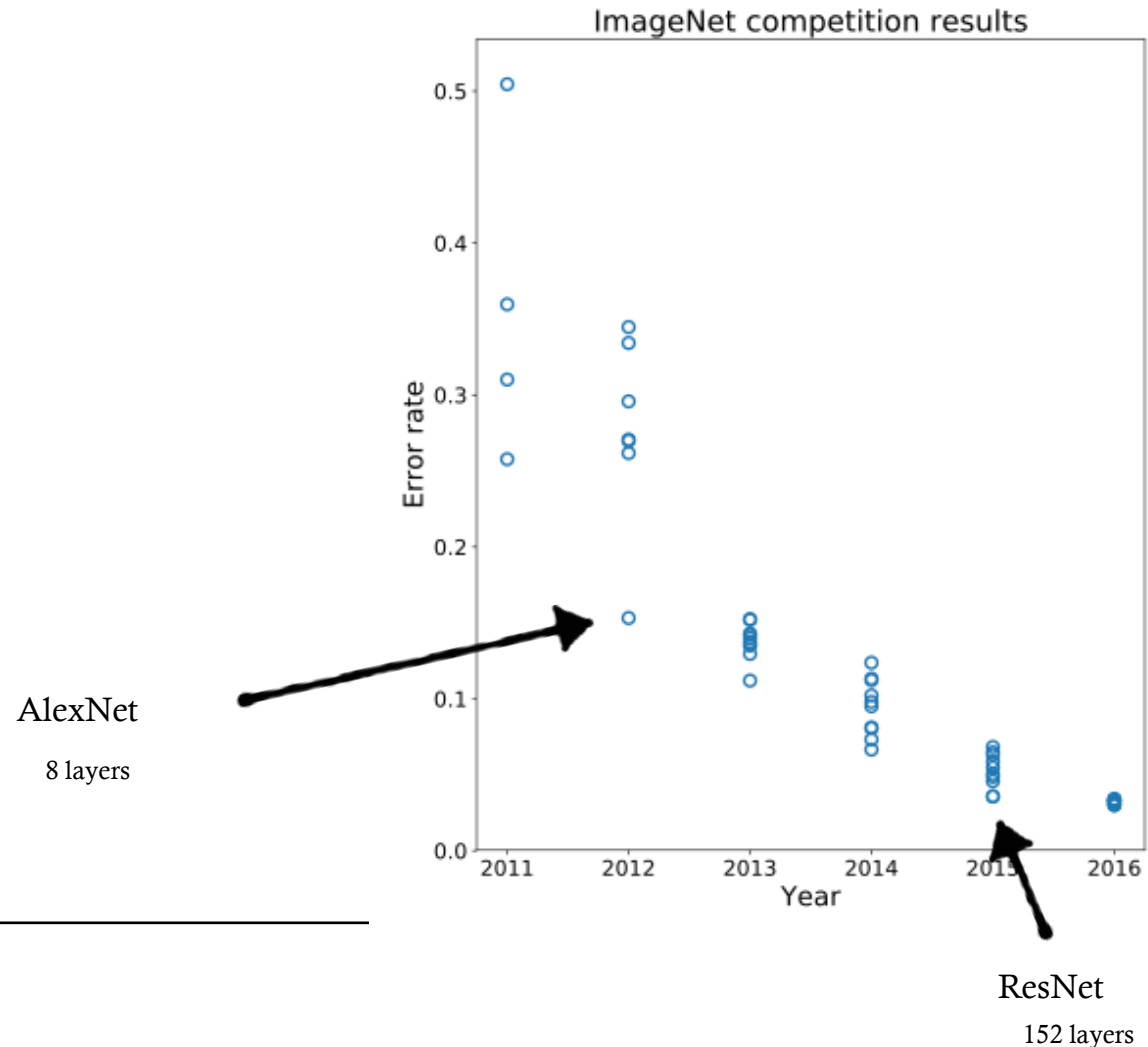
# IMAGENET

- Large visual database designed for object recognition research
  - ~14 million images
  - Hand-annotated (Mechanical Turk) into 20,000+ categories
  - Subset of the data is used in the ImageNet Large Scale Visualization Recognition Challenge (ILSVRC)
    - Help end AI winter
    - In 2012 AlexNet achieved a top-5 error rate more than 10.8 percentage points lower than the runner up
    - “Suddenly people started to pay attention, not just within the AI community but across the industry as a whole”  
(from *The Economist*)
-

---

# IMAGENET LARGE-SCALE VISUAL RECOGNITION CHALLENGE

- Subset of ImageNet Data
- 1.2 million training images
- 1000 classes



---

# MODERN CNN ARCHITECTURES

---

---

# VARIOUS CNN ARCHITECTURES

- We should think of the word “architecture” fairly literally
- It takes time, education, and practice to become an “architect”
  - True for architects that design buildings
  - True for architects that design CNNs
- We can learn from what others have done and other architectures that have been successful



---

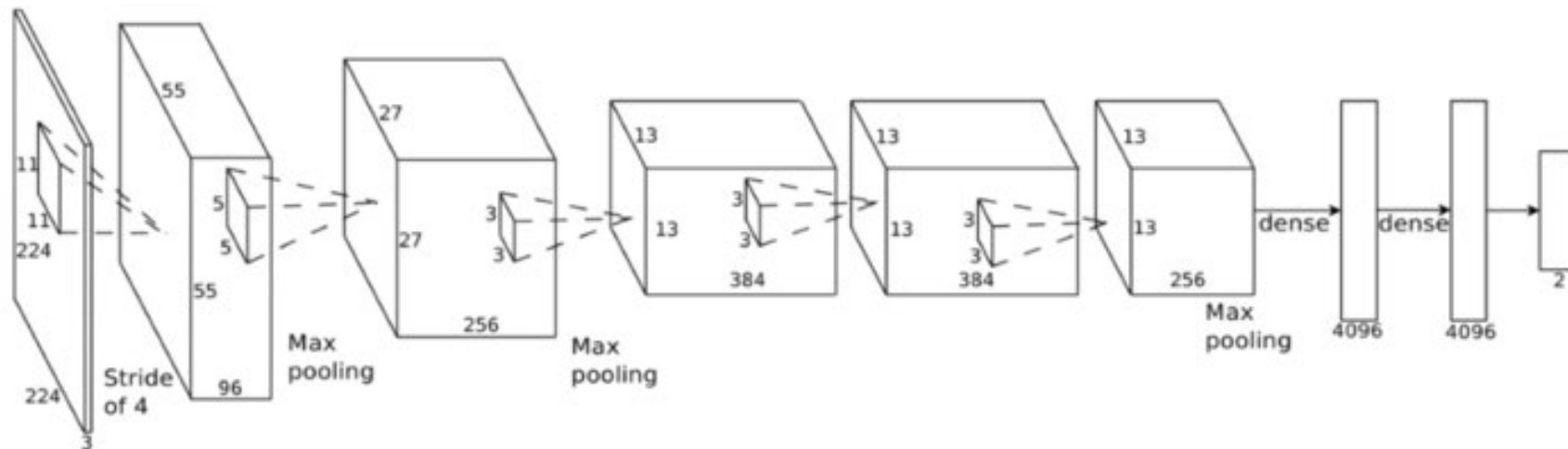
# INNOVATIONS THAT MADE DEPTH POSSIBLE

- ReLU (and its relatives)
  - Skip connections
  - Better weight initializations
  - Batch normalization
    - Normalize the inputs of each layer (helps with internal covariate shift, vanishing gradients, overfitting)
  - Dropout
    - Randomly dropout or “turn off” some of the neurons during training
    - Forces the network to have redundant representation of features
  - (Not an exhaustive list)
-

# ALEXNET (2012)

---

- Innovations
  - Deep convolutions
  - ReLu
- Also:
  - Dropout
  - Data augmentation
  - GPU





---

# RESNET (RESIDUAL NETWORK)

## Resnet

- Introduced by Microsoft researchers (He et al., 2015)
- Enables training of *very deep networks* by addressing vanishing gradients
- Add input directly to output of a few layers:

$$y = F(x) + x$$

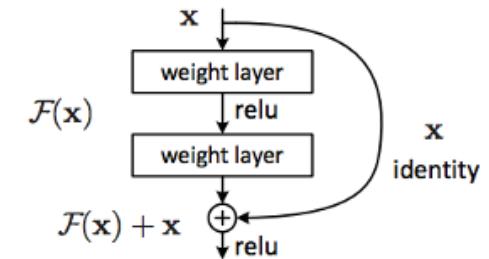
- Layers learn a *residual*  $F(x)$ : what to change, not the full transformation

## Intuition

- Easier to learn small adjustments than entire mappings
- If extra layers aren't useful  $\rightarrow$  learn  $F(x) \approx 0 \rightarrow$  behave like identity
- Network can “skip” unnecessary transformations

## Why does this help?

- Provides a direct path for gradients  $\rightarrow$  improves gradient flow
- Prevents degradation as depth increases
- Makes optimization of deep networks much easier



---

# CNN TRAINING

---

---

# TRAINING COMPONENTS

- Initialization
- Forward pass
- Compute loss
- Backpropagation
  - Update weights
- Repeat over mini-batches

Architectural hyperparameters:

- Kernel (filter) size
  - Stride
  - Padding
  - Number of filters
-

---

# CNN LAYER COMPONENTS

A convolutional layer is defined by:

- **Kernel size ( $K \times K$ )** → sliding window size
- **Stride ( $S$ )** → step size of movement
- **Padding ( $P$ )** → zeros added around border
- **Number of filters ( $F$ )** → number of feature maps
- Output spatial size:

$$\text{Output size} = \frac{N - K + 2P}{S} + 1$$

---

# KERNEL (SLIDING WINDOW)

**What does kernel size control?**

- Spatial pattern scale detected
- Parameter count
- Parameters per filter:

$$K \times K \times C$$

- Where  $C$  is the number of input channels (typically 1 or 3).
-

---

# KERNEL SIZE

- Larger kernels:
    - More parameters
    - Higher variance
    - More compute
    - Larger receptive field per layer
  - Smaller kernels (e.g.,  $3 \times 3$ ):
    - Fewer parameters
    - Deeper networks needed
    - Better regularization
    - More nonlinearities
  - This is why modern CNNs prefer stacked  $3 \times 3$  convolutions.
-

---

# STRIDE

- Stride = how far the filter moves each step.
  - $S = 1 \rightarrow$  dense coverage
  - $S = 2 \rightarrow$  downsampling
- Effect on output size:

$$\frac{N - K + 2P}{S} + 1$$

- Larger stride:
    - Smaller output feature maps
    - Fewer activations
    - Fewer gradient paths
    - Faster training
    - More aggressive information compression
  - Smaller stride:
    - More overlap
    - More gradient signal
    - Better fine-detail learning
    - Higher computation cost
-

---

# PADDING

Padding adds zeros around input borders.

- Types:
    - No padding
    - Same padding (preserve size)
  - Without padding:
    - Spatial size shrinks each layer
  - With padding:
    - Spatial resolution preserved
  - Without padding:
    - Border pixels used fewer times
    - Gradients weaker at edges
    - Spatial size shrinks quickly
  - With padding:
    - More uniform gradient flow
    - Deeper networks possible
    - Preserves spatial alignment
  - Padding helps stabilize training in deep CNNs.
-



---

# WEIGHT SHARING

## Fully connected (no sharing)

- If you flatten a  $224 \times 224$  image:
  - Input size  $\approx 50,000$  pixels
  - If next layer has 100 neurons
  - You need  $50,000 \times 100 = 5,000,000$  **weights**
  - Each neuron has its own independent weights.
- Suppose:
    - Kernel size =  $3 \times 3$
    - Input has 3 channels (RGB)
    - Number of filters = 64
  - Each filter has:
    - $3 \times 3 \times 3 = 27$  *weights*
  - Total parameters:
    - $27 \times 64 = 1728$  *weights*
  - Those weights for a filter are used **at every spatial location**
  - We do NOT learn different weights for different pixels
-

---

# USING PRE-TRAINED MODELS

---

---

# PRE-TRAINED MODELS

- Large models trained on massive datasets (ImageNet, web text, etc.)
- Learn *general-purpose representations*

## Why Use Them?

- Avoid training from scratch (expensive + data-hungry)
- Work well even with small datasets
- Faster convergence, better performance

## Which Layers?

- Initial layers → general features
  - Later layers → task-specific features
-

---

# TRANSFER LEARNING

- **Transfer Learning**
    - Keep lower layers (feature extraction)
    - Replace/train top layers for new task
    - Optionally fine-tune entire network
  - **Direct Use**
    - Apply the model as-is if the task is very similar
  - **Some Considerations**
    - Freeze layers when data is small
    - Fine-tune when data is larger
    - More similarity → less retraining needed
-